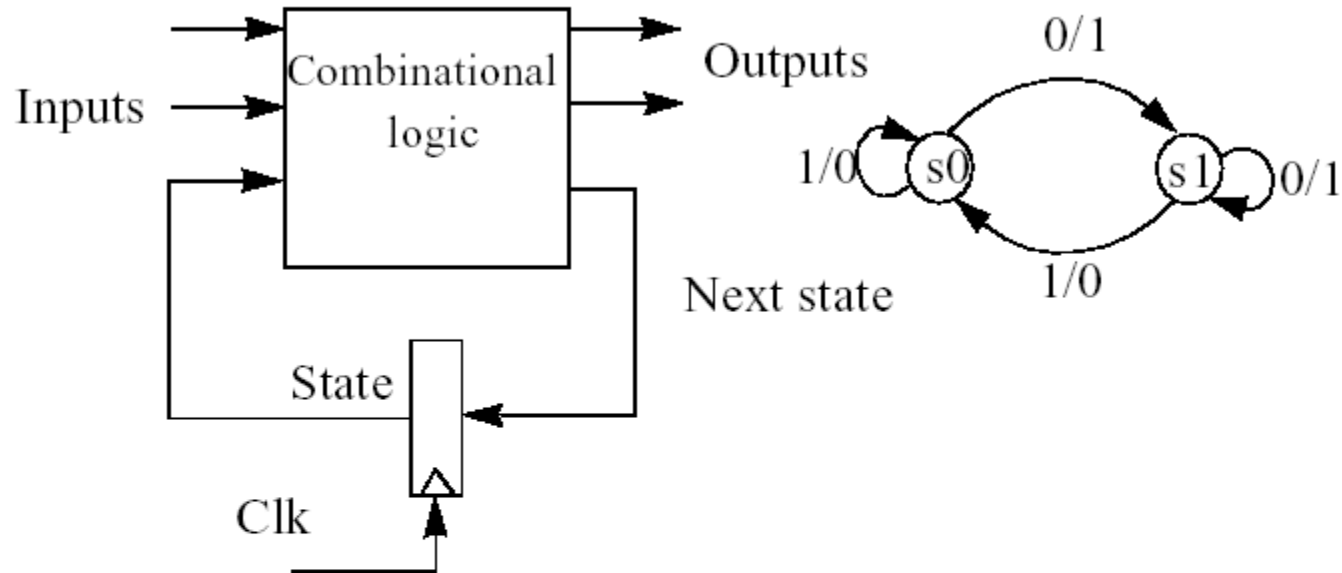
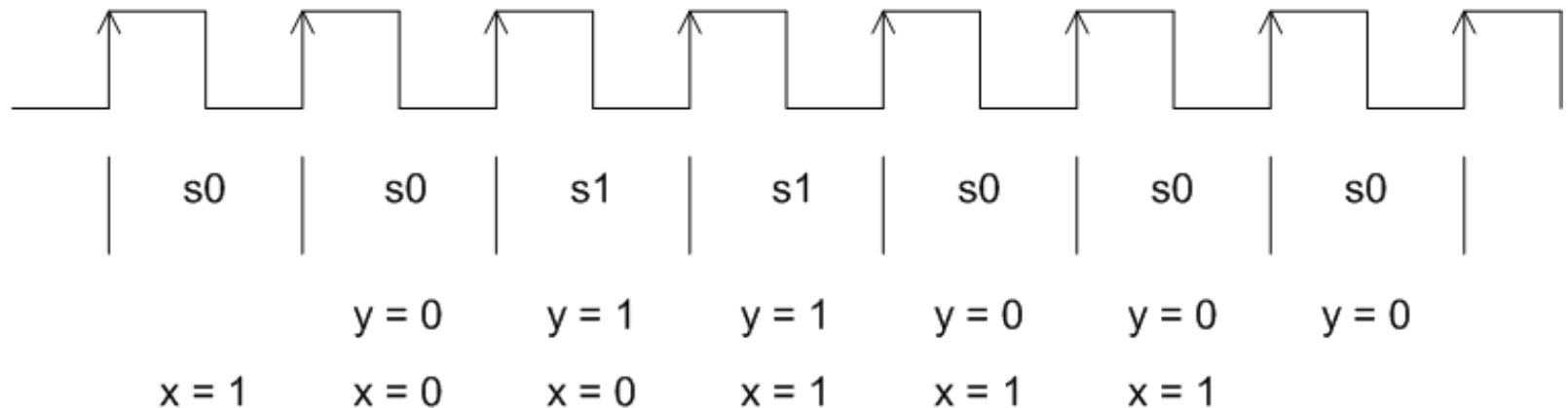


State Machines



- Basic components
 - combinational component: output function and next state function
 - sequential component
- Natural process implementation

State machine operation



FSM example

```
module fsm (x, clk, reset, y);  
input x, clk, reset;  
output y;  
localparam s0=0, s1=1;  
reg state, next_state, y;
```

```
// state flip-flops  
always @ (posedge clk)  
begin  
    if (reset==1)  
        state <= s0;  
    else  
        state <= next_state;  
end
```

```
/* combinational: next-state and  
outputs */
```

```
always @(x, state)
```

```
begin
```

```
    case (state)
```

```
        s0: if (x==1) begin
```

```
            next_state = s0;
```

```
            y = 1'b0;
```

```
        end
```

```
        else begin
```

```
            next_state = s1;
```

```
            y = 1'b1;
```

```
        end
```

```
        s1: if (x==1) begin  
            next_state = s0;
```

```
            y = 1'b0;
```

```
        end
```

```
        else begin
```

```
            next_state = s1;
```

```
            y = 1'b1;
```

```
        end
```

```
    endcase
```

```
end
```

```
endmodule
```

FSM example 2

```
module fsm (x, clk, reset, y);  
input x, clk, reset;  
output y;  
localparam s0=0, s1=1;  
reg state, nstate, y;
```

```
// state flip-flops  
always @ (posedge clk)  
begin  
    state <= nstate;  
end
```

```
/* combinational: next-state and  
outputs */
```

```
always @(x, state, reset)  
begin  
    nstate = state; // default  
    case (state)  
    s0: if (x==1) begin  
        nstate = s0;  
        y = 1'b0;  
    end  
    else begin  
        nstate = s1;  
        y = 1'b1;  
    end  
end
```

```
s1: if (x==1) begin  
    nstate = s0;  
    y = 1'b0;  
end  
else begin  
    nstate = s1;  
    y = 1'b1;  
end  
endcase  
  
if (reset==1) begin  
    nstate = s0;  
end  
end  
  
endmodule
```